



Lecture 2 — Claude Code in Practice

AI-Assisted Software Development and Intelligent Applications

Announcements

- 1. AI logging hook fixed for Windows.** It crashed silently on Windows (hardcoded `/tmp` path, wrong file encoding) — fixed upstream. Pull it in: switch your repo to the `upstream-sync` branch, click **Update fork**, then open a normal MR from `upstream-sync` into `main`.
- 2. Many P1 MRs aren't merging yet** — usually a **failing test** blocking the pipeline, not a review problem. Fix any failing tests first, then follow the [first-MR guide](#) to get reviewed and merged.

Today's Agenda

1. Essential Claude Code commands for coding sessions
2. Driving development with Claude Code in an unfamiliar codebase
3. The local development loop — test, format, lint, before you push
4. **Activity** — Coin Catcher: a TypeScript + p5.js practice project

Pre-Class Checklist — Start the Quiz

Before we start — did you complete the reading?

- **TypeScript basics** — types, interfaces, comparison to Java
- **TypeScript OOP** — classes, interfaces, access modifiers
- **p5.js** — the `setup()` / `draw()` loop, the canvas coordinate system
- **Vitest** — what a unit test is, arrange/act/assert

Today's practice project uses all four of these together.

From Java to TypeScript

Good news: the paradigm is familiar.

Java	TypeScript
<code>class Player { ... }</code>	<code>class Player { ... }</code>
static typing, <code>interface</code>	static typing, <code>interface</code>
access modifiers	same keywords

The syntax differs — but classes, interfaces, and types are concepts you already know.

p5.js: setup() and draw()

```
export function setup() {  
  createCanvas(600, 400);  // runs once  
}  
  
export function draw() {  
  background(20);          // runs ~60 times per second  
  // read input, update game state, draw to the canvas  
}
```

Same loop shape as a Java game's `init()` + game loop — p5 just calls `draw()` for you continuously.

Claude Code: Essential Commands

- `/help` — discover commands and what they do
- `@filename` — reference a file, Claude has precise context
 - *"Explain how `@src/Game.ts` decides when a coin is caught"*
- `/compact` — compress a long session's context
- `/clear` — discard the current session, start fresh
- `/context` — see current session's context window
- `/usage` — check token usage and subscription plan limits

Buzzer Checkpoint

**Launch Question 6 – discuss answers, then
continue**

Claude Code: Reviewing Edits

Claude proposes a diff before changing files — **read it before accepting.**

- What files does it touch — only the ones you expected?
- Does the change match what you asked for?
- Is there a smaller, more targeted way to make the same change?

Never blindly click Accept. If something looks off, reject and clarify what you meant.

Buzzer Checkpoint

**Launch Question 7 – discuss answers, then
continue**

Claude Code: Running Bash Commands

Claude Code can propose and run terminal commands on your behalf – installing packages, running tests, starting a dev server.

- You approve each command before it runs (in most permission modes)
- Ask it directly: *"Run the test suite and tell me what fails"*
- Useful for the local dev loop: let Claude run `npm run test`, `format`, `lint` and fix what it finds

Driving Development in Unfamiliar Code

When opening a codebase you didn't write:

1. **Ask before touching anything** — *"Overview this structure"*
2. **Trace the behavior you care about** — *"What happens in one frame?"*
3. **Ask about anything that looks strange** first
4. **Then** propose your change and iterate

Today's Activity, Part 1 — the Code

`src/format.ts` pads a score to 4 digits for the scoreboard:

```
export function formatScore(score: number): string {  
    return score.toString().padStart(4, "0");  
}
```

`formatScore(7)` → `"0007"`

`formatScore(12345)` → `"12345"` (never truncates)

Buzzer Checkpoint

**Launch Question 8 — discuss answers, then
continue**

Today's Activity, Part 1 — the Matching Test

`tests/format.test.ts` — Arrange, Act, Assert, one outcome per test:

```
import { describe, it, expect } from "vitest";
import { formatScore } from "../src/format";

describe("formatScore", () => {
  it("pads a single-digit score to 4 digits", () => {
    expect(formatScore(7)).toBe("0007");
  });
});
```

Buzzer Checkpoint

**Launch Question 9 – discuss answers, then
continue**

The Local Development Loop

The same checks your CI/CD pipeline will run on every merge request — run them **locally first** so failures are caught on your machine, not in the pipeline.

```
npm run test      # vitest - all tests must pass
npm run format    # auto-fix style with prettier
npm run lint      # eslint - fix any errors before pushing
```

The habit: test → format → lint → commit → push.

Why Run It Locally First?

- Pipeline failures cost a round trip: push → wait → read failure → fix → push again
- Running locally gives you the same answer in seconds, before anyone else sees a red pipeline
- `format` can auto-fix most style issues for you — let it
- A clean local run doesn't guarantee a clean pipeline, but it catches most problems early

Buzzer Checkpoint

**Launch Question 10 – discuss answers, then
continue**

Activity — Coin Catcher

A small TypeScript + p5.js game — same stack and tooling as Project 1, much smaller.

Before touching any code: use Claude Code to explore `project/` — ask it to explain the game loop and how `isCaught()` works.

Activity — Two Parts

Part 1 — Make a failing test pass: `formatScore()` in `src/format.ts` isn't implemented yet — `tests/format.test.ts` currently fails, and the game throws until it's fixed. Implement it (yourself or with Claude) so all 5 tests pass.

Part 2 — Add your own feature: pick **one** small feature (a hazard, a power-up, difficulty scaling, visual feedback — or your own idea), implement it with Claude Code, and write at least one vitest test covering it.

After each part: run `npm run test`, `npm run format`, `npm run lint` — fix anything flagged — commit, and push.



Questions?

Next lecture: Claude Code for Brownfield Development — read the [Claude Code Overview](#) and [Claude Code Memory](#) docs before class